



Support Lecture 04

CCP6224-OOAD



AWT

- Abstract Window Toolkit
- **AWT** (Abstract Window Toolkit) is one of the **earliest GUI libraries** provided by Java to build **Graphical User Interface (GUI)** applications.
- It is a **set of classes** in Java used to create **window-based applications**, such as:
 - Buttons
 - Text fields
 - Windows (Frame)
 - Dialog boxes
 - Menus
 - Event handling
- It is part of Java's standard library in the package

Components of AWT

Component	Purpose
Frame	Creates a window
Button	Clickable button
Label	Displays text
TextField	Input field
TextArea	Multi-line input
Checkbox	Check box
Choice	Dropdown list
Panel	Container for components

Example

```
import java.awt.*;

public class MyAWTApp
{
    public static void main(String[] args)
    {
        Frame f = new Frame("My AWT Window");
        Button b = new Button("Click Me");

        b.setBounds(80, 100, 100, 30);
        f.add(b);

        f.setSize(300, 300);
        f.setLayout(null);
        f.setVisible(true);
    }
}
```

AWT (Abstract Window Toolkit): Summary

1. **AWT = First GUI for Java**

(It was the first GUI toolkit used in Java. In early Java versions: up to 1.02; AWT was the only option for building windows, buttons, and other graphical components.”)

2. **AWT requires java.awt.* package**

3. **AWT is Platform Dependent**

(This means they rely on the underlying operating system’s native components. So, the same Java AWT button might look different on Windows, macOS, or Linux.)

4. **AWT is Heavyweight** (Because AWT uses the native OS GUI components)

5. **AWT is good, but considered as of limited characteristics**



SWING

- ❑ Swing is a **GUI toolkit** for Java with additional features
- ❑ Introduced as part of **Java Foundation Classes (JFC)**.
- ❑ Designed to overcome limitations of AWT.
- ❑ **Pure Java implementation** (lightweight).
- ❑ Provides a rich set of GUI components.

Why Swing Was Created

AWT was **limited**
and **platform**
dependent.

Developers
needs

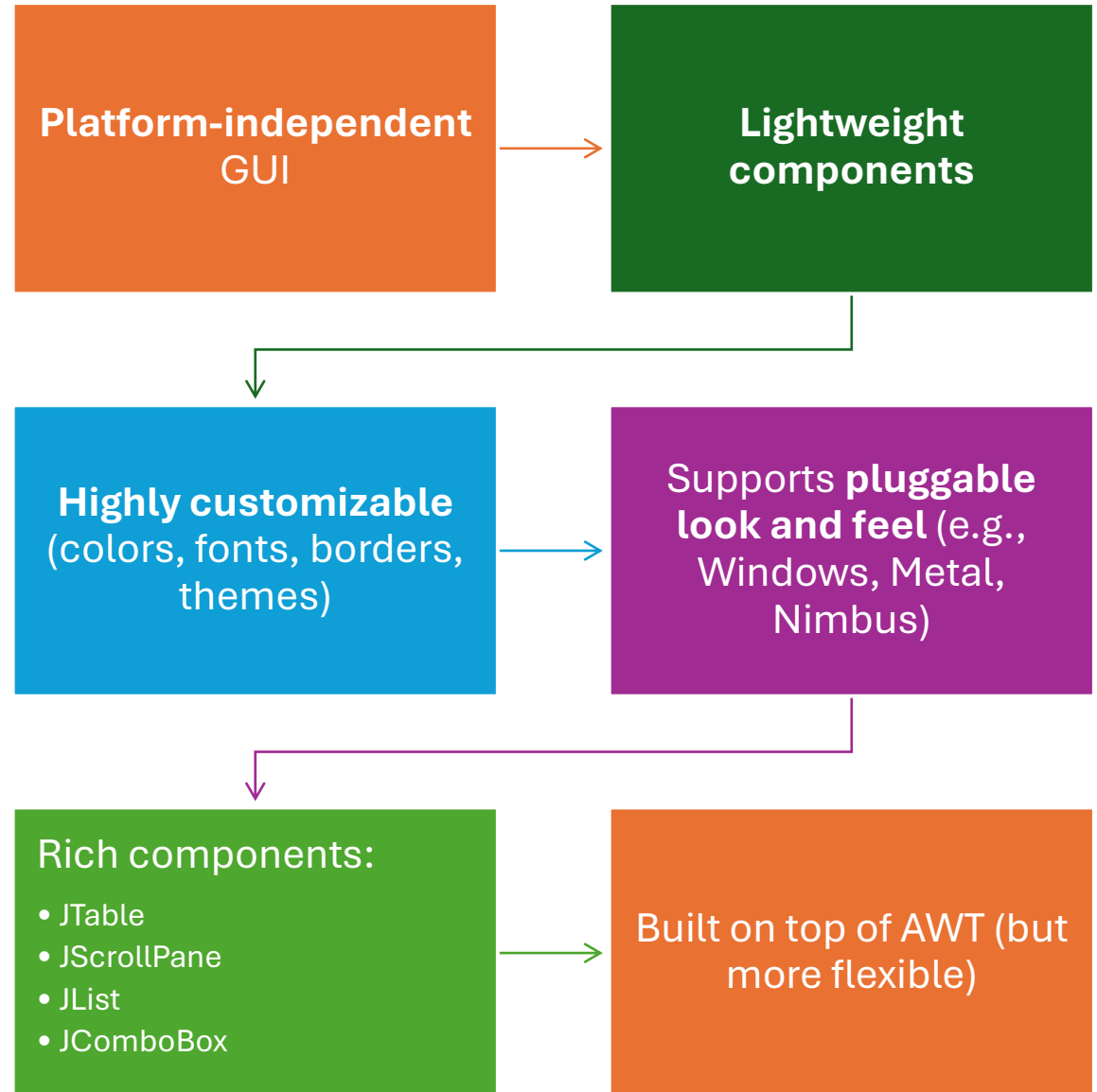
More
customizable
components

Consistent look
across platforms

Advanced GUI
elements

Swing replaced
most AWT use
cases.

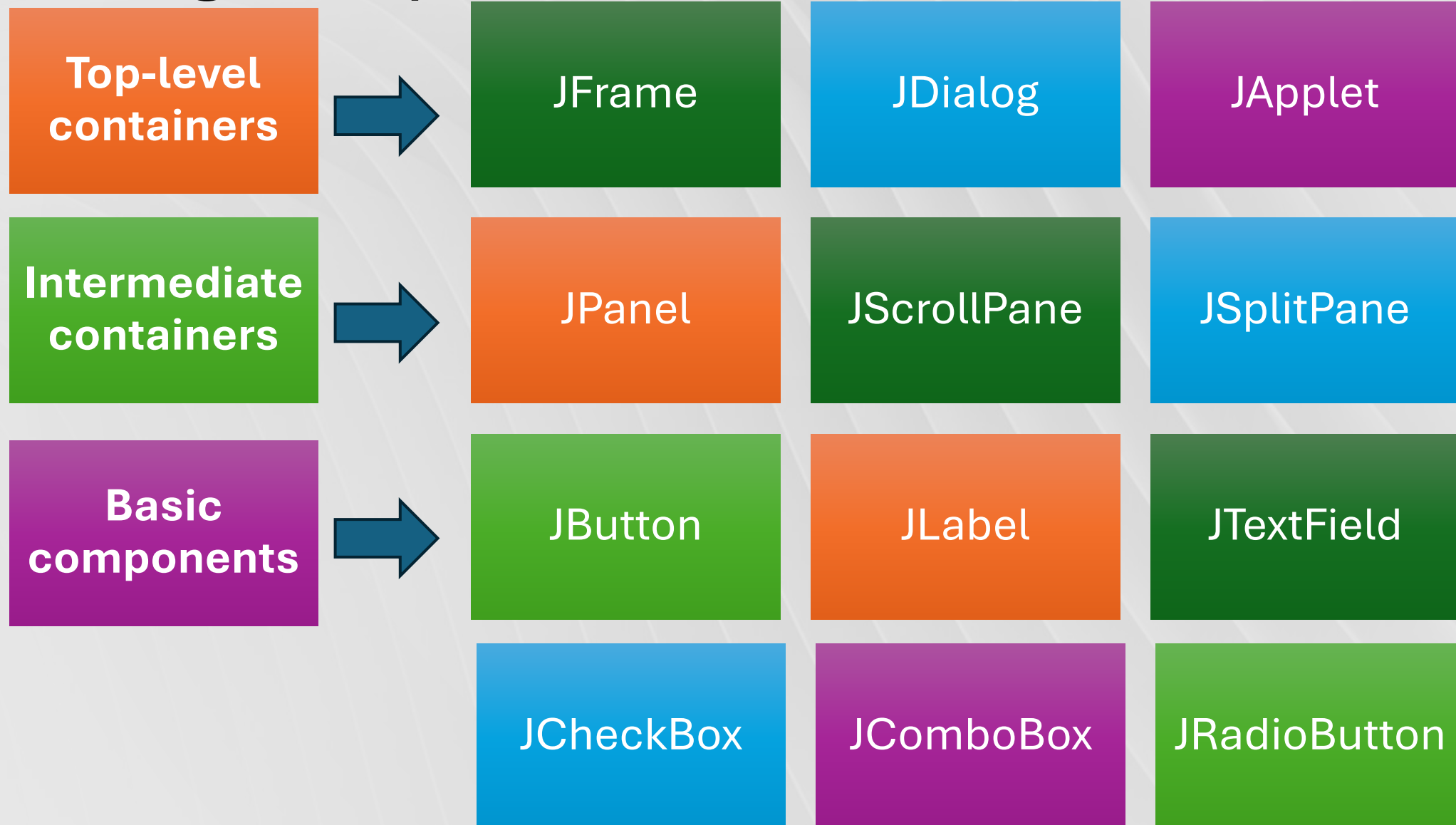
Key Features of Swing



Swing vs. AWT

Feature	AWT	Swing
Type	Heavyweight	Lightweight
Implementation	Uses OS GUI	Pure Java
Look & Feel	Depends on OS	Customizable
Components	Basic	Advanced & many
Flexibility	Low	High

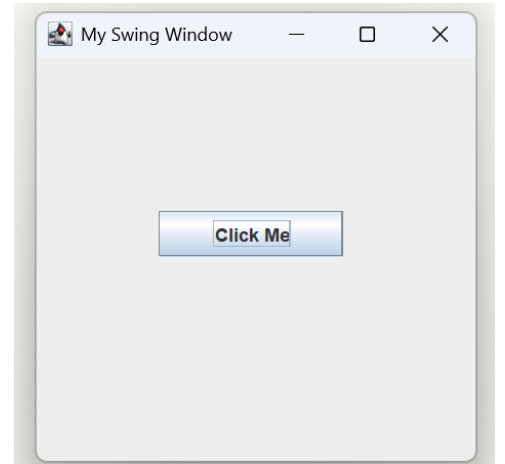
Swing Components Overview



Creating a Simple Swing Window

```
import javax.swing.*;
```

```
public class MySwingApp {  
    public static void main(String[] args) {  
        JFrame frame = new JFrame("My Swing Window");  
        JButton btn = new JButton("Click Me");  
  
        frame.setSize(300, 300);  
        frame.setLayout(null);  
  
        btn.setBounds(80, 100, 120, 30);  
        frame.add(btn);  
  
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        frame.setVisible(true);  
    }  
}
```



Fundamental Concepts in GUI Programming

Components and Containers

Components



Components are the basic GUI elements; these are the **objects that users can see and interact with.**



For example:

- Buttons
- Labels
- TextFields
- Checkboxes
- Scrollbars



Anything that you can click, type in, or visually see on the window is considered a ***component.***

Components cannot exist alone



- **Components cannot appear on the screen by themselves. They must always be placed inside a container.**
- To place a component inside a container, we use the **add()** method.

```
frame.add(button);
```

Fundamental Concepts in GUI Programming

Components and Containers

Containers

A container is simply something that can **hold components** or even other containers.



- Components = items (button, text field, etc.)
- Container = **box that holds these items**

Containers : Contd.

- **Functions of Containers**

“A container does two jobs:

- It **holds** components
- It **organizes** them using a specific **layout** (FlowLayout, BorderLayout, GridLayout, etc.)”

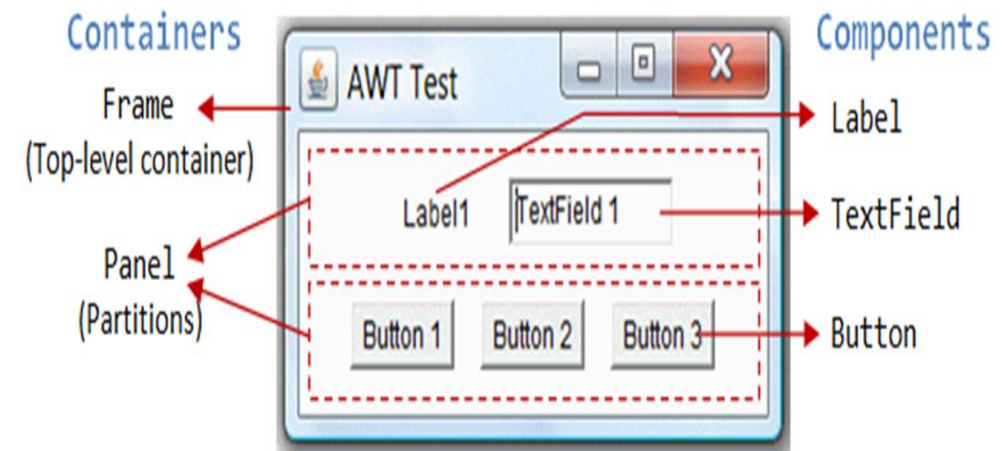
- **Containers Can Hold other Containers**

- In GUI design, containers can be **nested**.

Example:

- ✓ A Frame can contain a Panel
- ✓ A Panel can contain buttons
- ✓ A ScrollPane can contain another panel

This helps organize **complex user interfaces**



Types of Containers

(Top Level and Secondary Level)

Top-Level Containers

These are the windows that appear on the screen.
They are the outermost container where your GUI starts.

Examples in AWT:

Frame

Dialog

Their Swing equivalents:

JFrame

JDialog

Types of Containers

(Top Level and Secondary Level)

Secondary-Level Containers

These containers are placed **inside** top-level containers.
You usually use them to structure your interface neatly.

AWT Common examples:

Panel
ScrollPane

Swing equivalents:

JPanel
JScrollPane

These are extremely useful for grouping components and improving layout design.”

Simple Analogy (Components & Containers)



Think of **top-level containers** as your **house**.



Secondary –level containers are the **rooms** inside the house.



Components are the **furniture** inside the rooms.



We cannot put furniture directly in the air: We need a room.



Similarly, a component must be placed inside a container.

Summary

Components = basic GUI elements (button, text field)

Containers = objects that hold and organize components

Components can only exist inside containers

Two types of containers:

- Top-level: Frame, Dialog
- Secondary-level: Panel, ScrollPane

Event Handler

An **event handler** is a piece of code (usually a method) that runs automatically when a specific event occurs ; such as clicking a button, typing on the keyboard, closing a window, or moving the mouse.

An event handler is code that **responds to an action** performed by the user or the system.

Example:

- User clicks a button → event happens → event handler runs.
- User presses a key → event happens → event handler runs.
- User closes a window → event happens → event handler runs.

What is an Event?

An **event** is something that **happens** (usually **user interaction**):

User Action	Event
Button click	<i>ActionEvent</i>
Mouse click	<i>MouseEvent</i>
Key pressed	<i>KeyEvent</i>
Window close	<i>WindowEvent</i>

Types of Events

- **I/O events :** Events generated when input/output operations occur, such as reading from keyboard or writing to a file.
- **Handling events:** The process of writing code (event handlers) to respond to events triggered by the user or system.
- **Mouse events:** Events generated by mouse actions such as clicking, moving, dragging, or releasing the mouse.
- **Keyboard events:** Events generated when a key is pressed, released, or typed on the keyboard.
- **Components events:** Events triggered when a GUI component changes state, such as being resized, moved, shown, or hidden

Event Handler = Listener Method

- In Java, event handlers are written using **Listeners**.
- ❖ A listener is an **interface** with methods that automatically run when an event occurs.
- ❖ Examples:
 - ✓ **ActionListener**
 - ✓ **MouseListener**
 - ✓ **KeyListener**
 - ✓ **WindowListener**

Button Click Event Handler

```
button.addActionListener
( new ActionListener()
{
    public void actionPerformed (ActionEvent e)
    {
        System.out.println ("Button was clicked!");
    }
}
);
```

Window Closing Event Handler

```
frame.addWindowListener(new WindowAdapter()  
{  
    public void windowClosing(WindowEvent e) {  
        System.out.println("Window is closing");  
        System.exit(0);  
    }  
});
```

Layout Managers

- ❖ A **Layout Manager** in Java (AWT/Swing) is a **special object that controls how components (buttons, labels, text fields, etc.) are arranged inside a container** such as a JFrame, JPanel, Frame, or Panel.
- ❖ It decides **where each component will appear** and **how much space it will take**.
- ❖ A Layout Manager automatically arranges GUI components inside a window or panel.

Why Layout Managers are Needed?

If we don't use a layout manager:

Layout managers solve these problems by organizing components intelligently.

components may overlap

resizing the window breaks the UI

UI looks different on different screen sizes

difficult to maintain

Without Layout Manager

We must manually set position & size

```
frame.setLayout(null);
```

```
button.setBounds(50, 100, 100, 30);
```

With Layout Manager

Java arranges items automatically

```
frame.setLayout(new FlowLayout());
```

```
frame.add(button);
```

Layout Manager : Easy Concept

Exam Hall Seating Arrangement

Teacher = Layout Manager
Students = Components

When students enter the exam hall:

- Teacher arranges them row-by-row → **GridLayout**
- Every desk has equal space → **GridLayout behavior**
- No student decides their own position → **Layout Manager controls arrangement (ie Seating Plan)**

A Layout Manager is like a person who decides where each item should be placed neatly, without items overlapping.

Common Layout Managers

- ✓ **FlowLayout** : left-to-right (like text)
- ✓ **BorderLayout** : 5 positions: North, South, East, West, Center
- ✓ **GridLayout** : table-like rows × columns
- ✓ **BoxLayout** : vertical or horizontal arrangement
- ✓ **CardLayout** : switch between screens
- ✓ **GridBagLayout** : most flexible but difficult

Component event Listener

A **Component Event Listener** is a **listener** in Java that detects and responds to changes in a component's state, such as when it is moved, resized, shown, or hidden.

An event listener works like your brain.

It constantly watches for actions (like mouse clicks or key presses), and when something happens, it immediately reacts by running the appropriate code.

Human Body	Java Event Model
Sense organs detect actions	Event source detects events
Brain listens for signals	Event listener listens for events
Brain reacts (move hand, speak)	Event handler method executes action

Action listener

- An **ActionListener** is a Java event listener that **responds to *action events***, such as when a user clicks a button, selects a menu item, or presses **Enter** in a text field.

When is ActionListener used?

- ✓ Button click
- ✓ Menu item selection
- ✓ Pressing Enter in a text field
- ✓ Clicking a checkbox or radio button

```
button.addActionListener(new  
    ActionListener() {  
        public void  
        actionPerformed(ActionEvent e) {  
            System.out.println("Button clicked!");  
        }  
    });
```

Component Event Listener vs Action Listener

Feature	Component Event Listener	Action Listener
Purpose	Detects changes in a component itself (move, resize, show, hide).	Detects user actions like button clicks or pressing Enter.
Event Type	Component events	Action events
Triggered When	Component is moved, resized, shown, hidden.	User performs an action (clicks a button, selects menu item).
Common Interface	ComponentListener	ActionListener
Typical Uses	Responding to UI layout changes, resizing windows, tracking component visibility.	Handling button clicks, menu selections, form submissions.
Who triggers it?	The GUI component itself (system-generated events).	The user (user-generated events).
Key Methods	componentMoved(), componentResized(), componentShown(), componentHidden()	actionPerformed()
Example Component	Frame, Panel, any resizable component	Button, MenuItem, TextField
Simpler or Complex?	More technical (less used by beginners)	Very simple and most common event listener
Analogy	Like detecting when a door moves, opens, or closes.	Like detecting when someone presses a doorbell.

JavaFX

JavaFX is a modern Java framework used to build **rich, interactive, and visually appealing GUI applications** using graphics, controls, animations, and media.

Key Features

- **Modern GUI Toolkit** (replaces Swing for new projects)
- Supports **CSS styling** (like web design)
- Built-in support for **2D/3D graphics, animations, charts, video, and audio**
- Uses **FXML** (XML-based UI design) for cleaner interfaces
- Provides a rich set of controls like buttons, tables, sliders, date pickers
- Fully **cross-platform** (Windows, Mac, Linux)

public class HelloWorld extends Application

It means our class (HelloWorld) is inheriting from the JavaFX Application class.

The Application class is the **base class** for all JavaFX apps.

```
public class HelloWorld extends Application {  
  
    @Override  
    public void start(Stage stage) {  
        Button btn = new Button("Hello JavaFX");  
        Scene scene = new Scene(new StackPane(btn), 300, 200);  
  
        stage.setScene(scene);  
        stage.show();  
    }  
  
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```

When you **extend Application**, you are telling Java:

👉 “This class is a **JavaFX program**.”

Override the start(Stage stage) methodLaunch the UI through Application.launch()

Thank You

